

Igel Aergen - Adil Oubninte, Noé Vincent

Temps dévoué au projet :

- Adil Oubninte: env. 30 heures
- Noé Vincent: env. 20 heures

I. Introduction

Igel Aergen est un jeux de plateau se jouant à plusieurs (2 à 6 joueurs normalement), l'objectif ici à été de produire un programme C permettant de jouer à ce jeu au travers d'un seul et même terminal d'ordinateur ou, dans notre cas, en réseau.

Ce projet permet d'explorer différentes facettes et différents niveaux de la programmation C, allant du réseau à la gestion de structures abstraites. De plus, il nous aura permis d'utiliser et de nous familiariser encore un peu plus avec des outils permettant le bon développement d'un projet: la compilation automatisée avec CMake et la collaboration avec Git.

II. (Pas vraiment une) extension - Resizable Char Array

Objectif

Dans un souci d'économie de la mémoire, nous avons décidé d'implémenter les piles qui constituent les cases à l'aide de tableaux dynamiques. Ceci permet d'éviter d'allouer, pour chaque case, l'espace nécessaire à stocker les hérissons dans le pire cas possible, c'est-à-dire tout les hérissons sur la même case.

Cette extension n'en est pas vraiment une car elle ne peut être désactivée, c'est plutôt un choix d'implémentation que nous avons considéré comme suffisamment influent pour mériter sa place dans le rapport.

Réalisation

Un module `ResizableCharArray` a été créé, il permet le type `rca_t`. Celui-ci est utilisé pour stocker les hérissons situés sur une case. L'implémentation est classique, les tableaux sont initialisés de sorte à n'utiliser que peu de mémoire, l'espace alloué est doublé de taille à chaque dépassement de la capacité.

III. (Une toute petite) extension - Choix des positions de départ

It's not a bug, it's a feature !

Objectif

Cette extension vient initialement d'une erreur dans la compréhension du sujet. Nous avons finalement décidé de conserver cette fonctionnalité, le joueur a donc le choix de placer ses pions à la main ou de manière aléatoire (en fonction de la valeur de `PLACEMENT_RANDOM` dans `main.h`).

Réalisation

Dans le cas où cette extension est activée, on demande à chaque joueur de sélectionner les lignes où ils souhaite placer ses hérissons.

IV. Principale extension - Mode multijoueur en réseau

Afin d'activer, ou non, le mode multijoueur en réseau, deux options sont disponible:

- lancer le programme et suivre les instructions affichées
- passer comme argument du programme l'entier correspondant (1: mode classique, 2: mode serveur, 3: mode client).

Objectif

Afin de permettre un mode multi-joueur plus confortable, pour les joueurs, nous avons décidé d'implémenter une solution permettant de jouer à plusieurs depuis différentes machines (une par joueur) sur le même réseau local (ou sur des réseaux différents mais dont des ports spécifiques ont été ouverts).

Réalisation

Pour ce faire, une architecture client-serveur à été choisie. L'essentiel du code soutenant cette architecture se situe dans le module `multi`, créé par nos soins. Notons que chaque machine stocke sa propre instance du plateau, celle-ci évolue avec les coups jouée par chaque joueur. Le serveur cherche donc à synchroniser les différents plateaux des différents clients.

Le client exécute la fonction `client()`, celle-ci prends comme argument une structure portant diverses informations de connection et paramètres de la partie.

Le serveur, lui, à un rôle un peu plus complexe. En effet, le serveur est aussi joueur, il exécutera donc en parallèle :

- la fonction `serveur()` permettant d'orchestrer les différentes phases du jeux et la synchronisation des plateaux entre les joueurs.
- la fonction `client()` connectée au serveur hébergé sur la même machine, permettant au propriétaire de la machine de jouer aussi.

La communication entre clients et serveur se fait au travers de sockets réseau, à l'aide du module `csapp.h`. Ainsi cette extension fait intervenir 2 composantes imprévues pour le projet: le gestion de sockets et le multithreading.

Protocole de communication

1. Phase d'initialisation:

- Le serveur envoie à chaque client connecté la requête `"who"`
- Ceux-ci lui répondent en indiquant leurs numéro de joueur (avec une commande `"im"`)
- Une fois que le nombre attendu de joueurs s'est connecté, le serveur envoie `"all_player_ok"` à chacun des joueurs
- S'ensuit alors un échange avec chaque client afin d'obtenir le placement de leurs hérissons. (Requête `"place"`, réponse et propagation `"placed"`)
- À la fin de cette phase, le serveur envoie `"start"` à chaque joueur.

2. Phase de jeu

- à chaque tour, le serveur envoie "play" au joueur concerné.
- celui-ci répond avec un message décrivant son coup avec une commande "moved".
- le serveur propage ce coup aux autres joueurs avec une commande "moved".
- un fois qu'un client a gagné, il propage une commande "exit".

Un commande correspond au type suivant:

```
struct commande{
    bool is_cmd;
    int id;
    int nb_args;
    bool auto_instancie;
    char* cmd;
    char** args;
};
```

Petite explication des champs:

- `is_cmd` indique si une réponse du client est attendue
- `id` indique le joueur concerné par la commande (si c'est un message du serveur comme "all_player_ok" : -1)
- `nb_args` indique la taille de `args`.
- `auto_instancie` indique si les strings fournit ensuite sont sur la pile ou le tas, permettant de savoir si elles doivent être libérées ou non.
- `cmd` indique le type de la commande parmi ("who", "im", "all_player_ok", "place", "placed", "start", "play", "moved", "exit")
- `args` contient des informations complémentaires à la commande (exemple, les lignes dans lesquelles sont initialement placés les hérissons d'un joueur, dans le cadre d'une commande "placed")

Exemple de message :

Placement des hérissons par le joueur 0, en réponse à une commande "place" envoyée par le serveur. Le joueur 0 place des hérissons aux lignes 0, 4 et 5.

```
commande_t c = {true; 0; 3; false; "placed"; {0,4,5}};
```

V. Synthèse

Observons tout d'abord que tout les modes de jeux fonctionnent (c'est déjà une victoire !) cependant, certaines observations sont nécessaires.

Remarquons le bilan mitigé de l'extension permettant le choix des positions de départ des hérissons. En effet, celle-ci ajoute une lourdeur considérable au jeu sans apporter un grand avantage ludique.

Aussi, le choix aléatoire des positions initiales des hérissons semble être d'assez mauvaise facture.

En effet, dans le mode classique, les hérissons, bien que placés sur des lignes au hasard, le sont dans un ordre prédéterminé (1er herisson du joueur 0, 1er herisson du joueur 1, etc...), ainsi le dernier hérisson du dernier joueur se trouvera toujours en haut de sa pile. Ainsi, ce mode de placement des hérissons ne suit pas tout à fait la demande d'aléatoire. La solution aurait été de tirer au hasard une permutation des joueurs afin que le i^{eme} empilement (celui du i^{eme} herisson de chaque joueur) se fasse dans un ordre aléatoire, mais cela n'a pas été implémenté.

Cependant, il y a pire, dans le mode multijoueur, bien que les lignes sur lesquelles les hérissons sont placés soient tirées au hasard, l'ordre dans lequel ces herissons sont empilés sur les cases correspond à l'ordre croissant des identifiant des joueurs. Ainsi, le dernier joueur trouvera toujours ses hérissons au dessus de ceux des autres (tout les hérissons du joueur 0, tout ceux du joueur 1, etc..). Une implémentation centralisée, utilisant des permutations tirées au hasard, aurait été préférable mais n'a pas été mise en place.

De plus l'extension permettant le mode multijoueur en réseau créé un certain nombre de fuites de mémoire dû à l'hétérogénéité des modes d'instanciation des commandes. Une procédure unique et standardisée aurait été préférable mais elle n'a pas été implémentée.

VI. Bibliographie

- Module `csapp.h`, Carnegie Mellon University, [Lien vers csapp.c](#), [Lien vers csapp.h](#)

Ce module a été utilisé selon les conseils de Guillaume Didier, enseignant le Réseau au sein du Module SYS en 1ère année du département d'informatique de l'ENS de Rennes. Ce module a permis de simplifier grandement la gestion des sockets réseau et de ce fait, a permis la communication entre clients et serveur.

- Cours de Guillaume Didier, SYS-L3 INFO ENS Rennes

Ce cours a été utile dans la compréhension et la mise en pratique du module `csapp` cité plus haut.

- Variadic Function in C Programming, Loo Kian Selong, The Startup, [Lien vers l'article](#)

Cet article a permis de maîtriser les fonctions de `stdarg.h` utilisée notamment dans les fonctions gérant les commandes et requêtes échangées entre le serveur et les clients.

- StackOverflow

Forum utilisé dans des cas marginaux pour traiter des bizarreries de C, en particulier pour le traitement des strings et buffers.